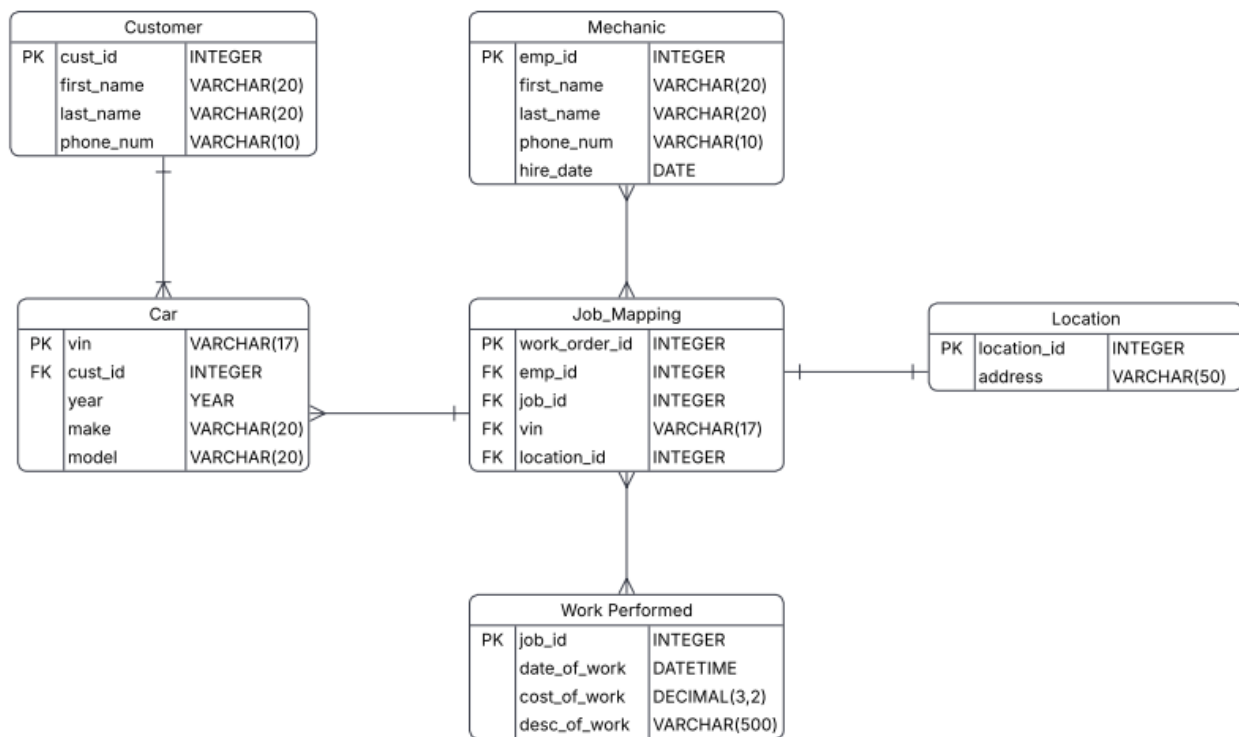


The Shop Manager System is a lightweight app that allows a mechanic shop owner to manage systems relating to their shops. This includes mechanics, customers, jobs performed, locations, and cars. This allows owners of smaller/independent shops to have a centralized location to track business information without the bloat that commercial/enterprise software might have.



The above ERD shows the entities and their relationships within this system. The assumptions are as follows:

- A car can only have one owner (Customer), but a customer can have many cars.
- One job is tied to one car, but a car can have many jobs performed on it.
- Multiple mechanics can work on multiple jobs.
- One job can only be performed at one location.

Primary keys are set to be standard numerical keys, aside from Car. Car has the VIN set as its primary key since it is a unique identifier from the factory. Most foreign keys are handled by the Job_Mapping junction table. Car has the cust_id foreign key to have its owner assigned to it.

The design of this was left in First Normal Form. While it could've been Second Normal Form if Car was not dependent on cust_id, there was no real need to add the extra effort.

The "Job_Summary" view joins Job_Mapping, Work_Performed, Mechanic, and Car into one result. It displays the workorder id, mechanic's name, the car make/model, and cost of the job. This is a high level way to see what work was performed on what car.

Four most important queries:

1. LEFT JOIN: Cars with their work order history
 - a. Shows every car belonging to a customer alongside any associated work order, using NULL to flag cars that haven't had service yet.
2. GROUP BY with HAVING: Mechanic totals over \$200
 - a. Sums total billed work per mechanic to those exceeding \$200, giving a quick view of the workload of a mechanic.
3. Subquery: Customers above average job cost
 - a. Displays customers whose job cost exceeded the shop's average.
4. View query: Job_Summary over \$100
 - a. Displays a report of higher cost jobs using the Job_Summary view instead of the four-table join. Used to sort out smaller maintenance-related jobs such as filter changes.

The database is completely manageable from the front end interface. The front end will be able to perform standard CRUD operations that a shop owner would want to use. It also has the capability to run reports so that an owner can quickly see relevant business information.